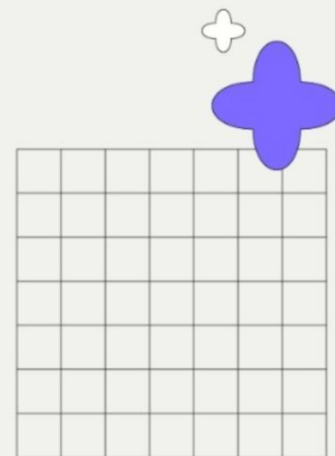
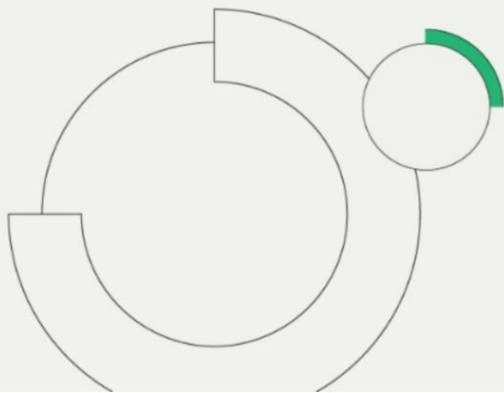
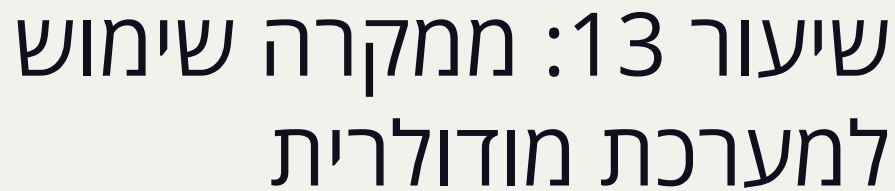




**שיעור 13 - יצירת מערכת ללא AI -
מקרי שימוש**



בונים את מערכת התקלות ב־Apps Script

קורס מיישם AI, האקריו



איפה אנחנו במסע

1

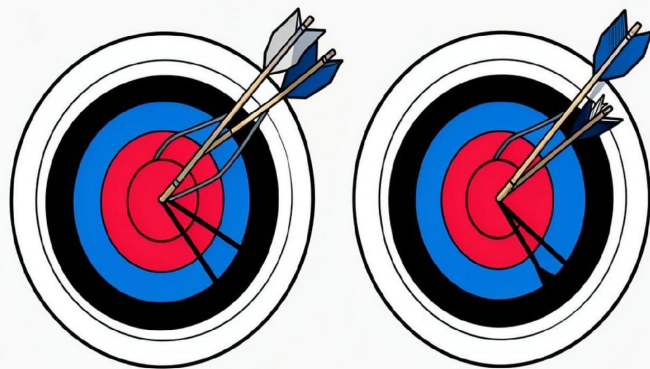
שיעור 12: סגירת שלב
ההוכחה
אב טיפוס מאומת, שפת
שלוש השכבות, וסט הכלים
החינמי.

2

אחרי אב טיפוס מאושר
שלב החלטות הפיתוח.

3

חלק ג של ה-PRD
ארכיטקטורת מערכת- סעיף 14: מקרי שימוש ותהליכים.



שתי מטרות היום

יישומית

נבנה יחד את שלד מערכת התקלות ב־Apps Script,
מודולרי מהצעד הראשון

מתודולוגית

טבלת מקרי השימוש שהופכת למפרט הבנייה

בסוף המפגש: כתובת שעובדת

על מה מתבססים: לא כותבים תרחישים מאפס

הגישה הקלאסית:

סיפור משתמש כלל שלושה חלקים:

- as a - פרסונה
- I WANT - הפעילות הנדרשת
- SO THAT - התוצאה/ הסיבה קריטריוני הצלחה מדידים.

מקרה שימוש: מסמך שנכתב מאפס:

- שחקן
- תנאים מוקדמים
- זרימה ראשית
- חלופות וחריגות
- איכויות.

המתודולוגיה שלנו

התרחישים כבר קיימים בסיפורי המשתמש המורחבים.

סעיף 14 ב-PRD - מוסיף את החלטות הפיתוח.

מקרה השימוש:

- הרחבה הנדסית לסיפור המשתמש
- שכבה מעל התרחישים

שכבת ההחלטות: שואלים חמש שאלות לכל סיפור משתמש

לפי סעיף 14 של ה-PRD, לכל סיפור משתמש עונים על:

1

מה מפותח בקוד רגיל, ללא AI?

2

היכן נדרש AI, באיזו דרגת מודל, פיצ'ר, ולמה?

3

מהם הממשקים ונקודות ההתממשקות?

4

לאיזה מודול/ פיצ'ר במבנה המערכת היכולת שייכת?

5

אילו חריגות מערכתיות התגלו באימות אב הטיפוס?



חריגות מערכתיות - התנהגות המערכת בתנאי הפעלה חריגים

הגדרה: התנהגות נדרשת כאשר הנסיבות סביב הבקשה חריגות, ולא תוכן הבקשה עצמה

- **קטגוריות אפשריות:** עומס במקביליות, מכסה שנוצלה, תלות חיצונית שאינה זמינה, נפח נתונים, קלט בגודל חריג...
- **קביעת מדיניות לחריגות:** מי מקבל קדימות, מה נדחה, מה ממשיך לפעול במתכונת המצומצמת, מה עושים עם מה שנדחה
- **הכלל במערכת:** פעולות הליבה נשמרות בכל מצב.
- **דוגמה:** דיווח נקלט גם כשהמכסה נוצלה, גם כשספק ה-AI אינו מגיב, וגם בשעת עומס
- **מה שנדחה:** עיבוד כבד שרץ ברקע, נכנס לתור של עיבוד עתידי. שימוש שוטף מקבל קדימות

שלוש בדיקות לכל החלטה

בדיקת הצורך/ רצון
את איזה סיפור משתמש מה-PRD היכולת משרתת? אם אין
סיפור משתמש, יוצרים אותו!

בדיקת החלופה הפשוטה/ הזולה
קוד רגיל לפני AI, מודל זול לפני יקר, פתרון קיים לפני פיתוח.

בדיקת העלות המתמשכת
כמה בחודש כולל תחזוקה, ומי הבעלים?

יכולת שנפלה בבדיקה מקבלת שורת דחייה מתועדת (ADR) עם
שלב היעד והנימוק. דחייה מתועדת היא תוצר לגיטימי כמו יישום.



שכבות ומודולים: שני חיתוכים לאותה מערכת

לרוחב - השכבות שכבר למדנו
מציג, מחליט, זוכר

לאורך - המודולים
מי אחראי על מה

בתוך כל מודול - הפיצ'רים
מה המשתמש מקבל

שכבה עונה איזה סוג עבודה זה, מודול עונה על מה זה אחראי,
פיצ'ר עונה מה מקבלים



מודול: מבנה עם אחריות ספציפית



אחריות אחת ברורה

☐

ממשק קבוע שדרכו פונים אליו

☐

חוסר תלות במודולים אחרים

☐

מודול חוצה את שלוש השכבות

☐

יש לו מסך משלו, החלטות משלו, נתונים משלו

צריך לבדוק

שאפשר להחליף אותו, כל עוד הממשק נשמר

במערכת שלנו, שישה מודולים



דשבורד

התמונה הכוללת והערך שנחסך.
סיפורים 4 ו-7



מעקב ומונה אישי

מה קרה לדיווח שלי, וכמה
איתורים צברתי. סיפורים 3,5,6



דיווח

העובד מדווח, והדיווח נקלט.
סיפורים 1 ו-2



הגדרות

ספקים, היררכיה ומקורות



ניתוח

תקלות חוזרות ומגמות. סיפור 8



עוזר חכם

מענה לשאלות עובד מתוך
הנהלים

הפיצ'רים שבתוך מודול

בתוך כל מודול יושבים פיצ'רים שנדרשים לביצוע:

דוגמה - מודול הדיווח:

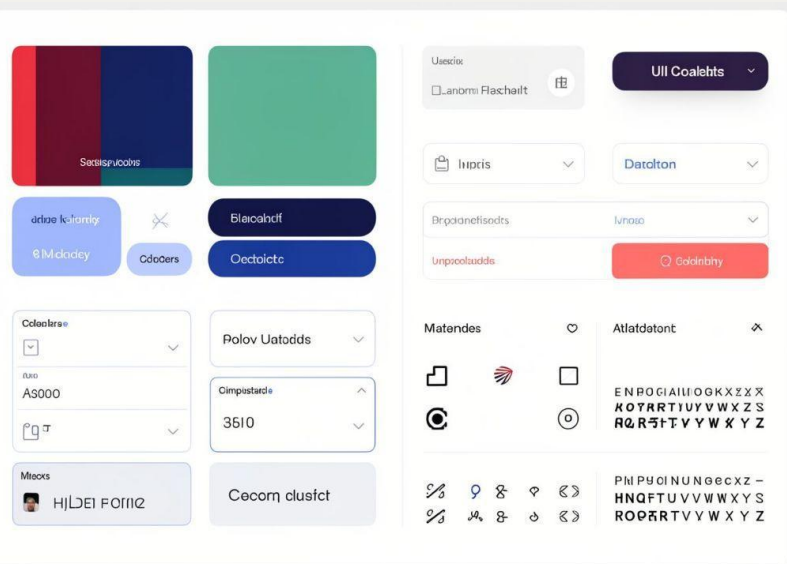
- שליחת דיווח
- ניסוח הדיווח מדיבור חופשי
- צפייה במצב דיווח קיים

The screenshot shows a complex web form with multiple sections and input fields. The form is organized into several columns and rows. At the top left, there are two large colored squares (red and green) with the text 'Satisfructus' below them. Below this, there are several buttons and input fields, including 'dirize khamiy', 'Blaicahdf', 'Cdoers', and 'Oectaictc'. In the middle left, there are two dropdown menus labeled 'Cdoalers' and 'Polov Uatocids'. Below these, there are two more dropdown menus labeled 'A5000' and '3510'. At the bottom left, there is a section labeled 'Miaors' with a profile picture and the text 'HJDEI FOMZ'. In the middle right, there are two dropdown menus labeled 'Birpianetisodts' and 'Unpooobuads'. Below these, there are two buttons labeled 'Goldriny' and 'Goldriny'. At the bottom right, there are two sections labeled 'Matandes' and 'Atlatdant' with various icons and text. The form is designed with a clean, modern aesthetic using a light gray background and blue accents.

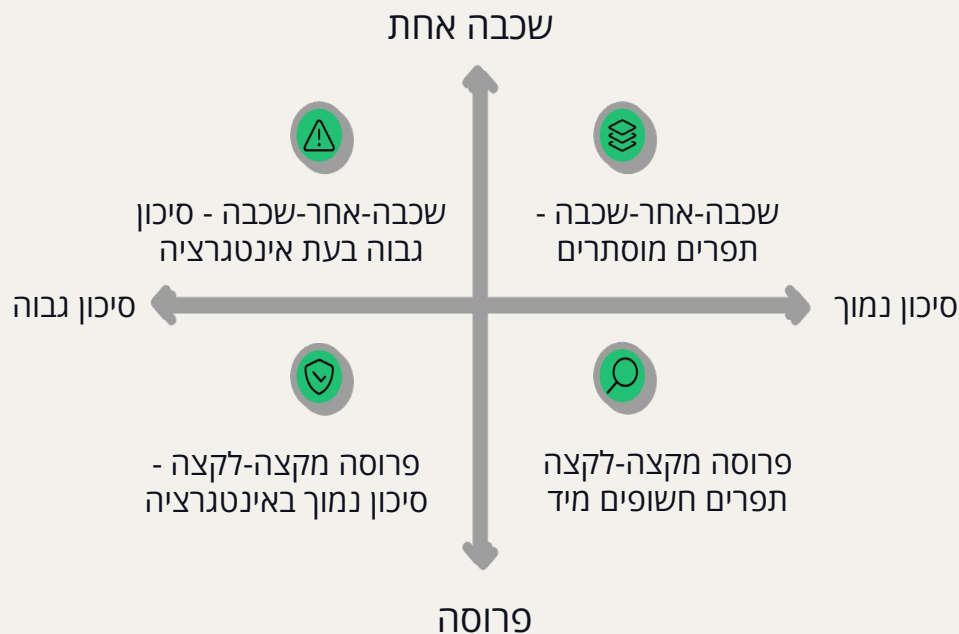
מודול העיצוב

מודול עיצוב אחד משותף לכולם:

מקור אמת יחיד לצבעים ולרכיבי הבסיס, כדי
שכל המסכים ייראו מערכת אחת



עיקרון הבנייה: שכבה אחר שכבה או פרוסה מקצה לקצה?



למה לא לבנות כל שכבה בנפרד?

הסיכון נמצא בתפרים שבין השכבות:

- הרשאות
- פורמט JSON
- פריסה
- מפתחות

פרוסה חוצה את כל השכבות - מטפלת בתפרים,
בכל תרחיש



מהי פרוסה?

תרחיש אחד, מהמסך ועד לבסיס הנתונים, חוצה את כל השכבות.

כמו פרוסת עוגה: חיתוך שטועמים בו את כל השכבות

הפרוסה שלנו

יצירת דיווח - מהמסך ועד שמירת הנתונים.

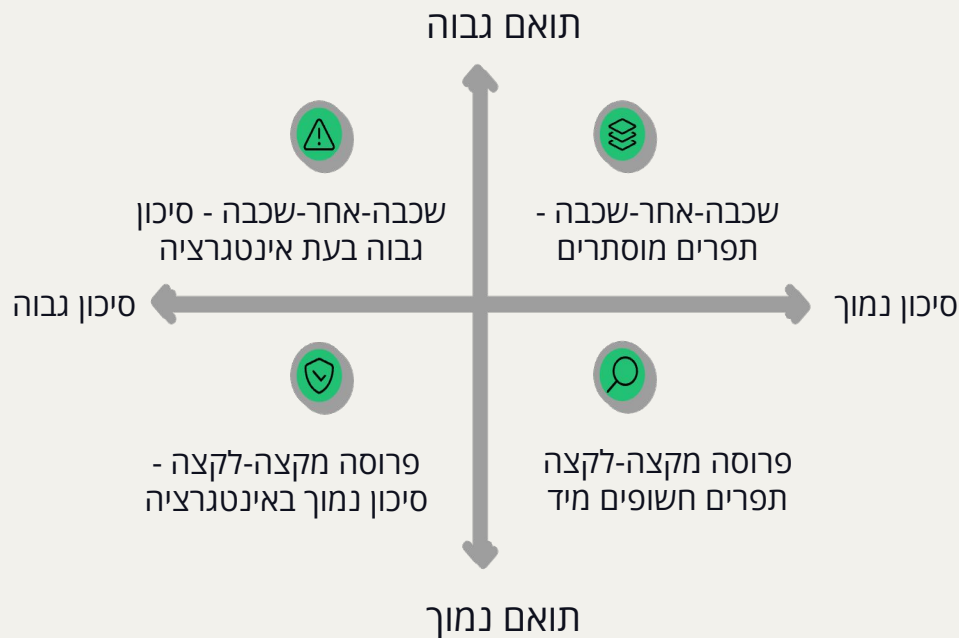
כשהיא עובדת

שלד המערכת (Walking Skeleton) - מערכת מינימלית

עיקרון הבנייה: שכבה אחר שכבה או פרוסה מקצה לקצה?

למה לא לבנות כל שכבה בנפרד?

הסיכון נמצא בתפרים שבין השכבות:



- הרשאות

- פורמט JSON

- פריסה

- מפתחות

פרוסה חוצה את כל השכבות -

מטפלת בתפרים, בכל תרחיש



מה זו פרוסה?

תרחיש אחד, מהמסך ועד לבסיס הנתונים, דרך כל השכבות.

כמו פרוסת עוגה: חיתוך שטועמים בו את כל השכבות בביס אחד.

הפרוסה שלנו

יצירת דיווח - מהמסך ועד הגיליון.

כשהיא עובדת

שלד מהלך (Walking Skeleton) - מערכת מינימלית שחיה.

מודול, פיצ'ר, פרוסה



פיצ'ר

יכולת - עונה מה
המשתמש מקבל

מודול

מבנה - עונה מי אחראי

פרוסה

מסלול - עונה איך בונים:
תרחיש אחד מקצה לקצה

בונים פרוסה אחת בכל פעם. הפרוסה מממשת מודול אחד או פיצ'ר אחד

פרוסות לעומת מונולית

פרוסה היא יכולת דקה בהיקף שלה. בתוכה יש ארכיטקטורה מלאה:



שמנתב doPost

העובר של

האורקסטרטור - מנתב

בקשות לפונקציות

הנכונות.



פונקציית Gemini
נפרדת

שירות ה-AI - מבודד
ומוחלף בקלות.



פונקציית כתיבה נפרדת

ה-Repository - כותב
לגיליון, ניתן להחלפה.

מבחן ההחלפה: בהמשך נחליף את הגיליון במסד נתונים - כמה פונקציות משתנות? אחת = מודולרי.

שרת, שירות, ומיקרוסרוויסים

שירות

תוכנה שרצה על שרת ועונה לבקשות בממשק מוגדר. במערכת שלנו: הפרויקט כולו הוא שירות אחד, עם כתובת אחת וממשק אחד.

שרת/ ענן

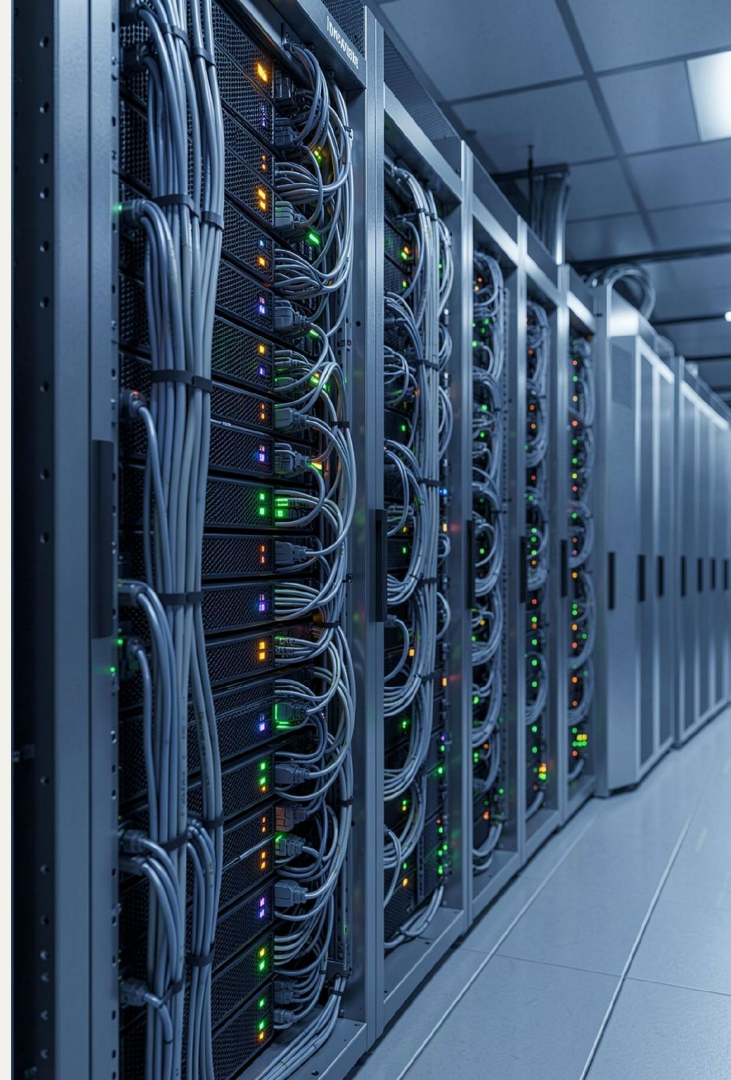
מחשב שמריץ את הקוד ועונה לבקשות. במערכת שלנו: Google מריצה את הקוד שלנו על השרתים בענן שלה. שרת בענן - מנוהל על ידי בעלי הענן.

מיקרוסרוויסים

שיטת בנייה שבה המערכת מפורקת לשירותים קטנים ונפרדים. כל שירות רץ בנפרד, מחזיק נתונים משלו, מתעדכן ומוחלף בנפרד, והשירותים מדברים זה עם זה באמצעות קישוריות.

ההשלכה על החלוקה

התוצר שיש לעדכן/ להחליף מונוליט, שכבה, מודול או מיקרוסרוויס



איך מחליטים: שני נתונים קובעים את החלוקה



גבול הפריסה

מהי היחידה הקטנה ביותר שניתן לעדכן ולהחליף בנפרד. במיקרוסרוויסים היחידה היא שירות בודד, כלומר פיצ'ר או פונקציה.

אצלנו המערכת כולה מתעדכנת ברמת המודול, ולכן יחידת ההחלפה היא מודול.

קצב השינוי

מודול שנדרש להתמודד עם תוספות תדירות מנוהל ברזולוציה של פיצ'רים. מודול יציב מנוהל כיחידה אחת.

שני הנתונים נבחנים לכל מודול בנפרד, וההכרעה נרשמת.

הכרעה 1: בונים מודול אחרי מודול

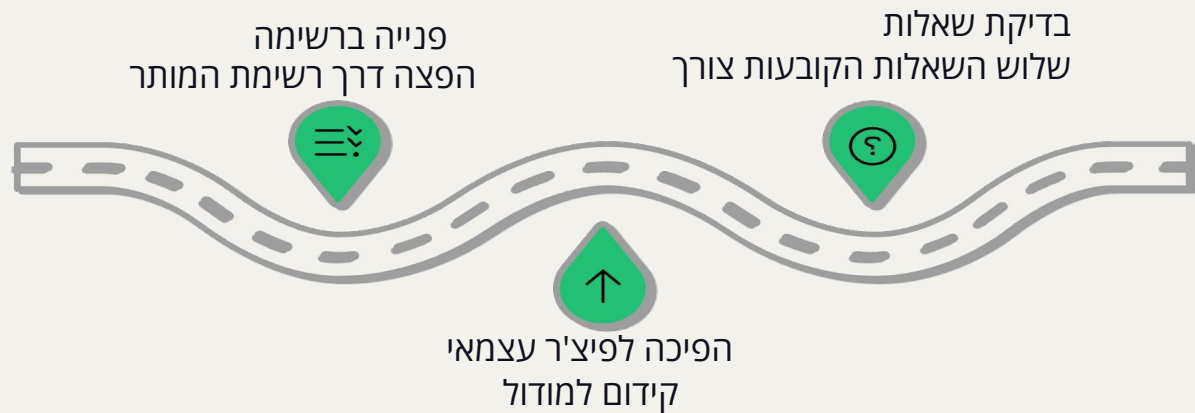
פיצ'ר כסדר עבודה פנימי
בתוך המודול, הפיצ'ר הוא
סדר העבודה: בונים פיצ'ר,
בודקים אותו, וממשיכים
לפיצ'ר הבא.

בנייה מלאה של מודול
כל מודול נבנה במלואו,
נבדק, ורק אז מתחילה
בניית המודול הבא.

יישום במערכת שלנו
מודול הדיווח ומודול
הניתוח צפויים לקצב שינוי
גבוה, ולכן ינוהלו
ברזולוציית פיצ'רים.

קובץ נפרד לכל פיצ'ר
בקצב גבוה
מודול בקצב שינוי גבוה
מקבל קובץ נפרד לכל
פיצ'ר. תוספת חדשה
מוסיפה קובץ, והקבצים
הקיימים נשארים במקומם.





הכרעה 2: פיצ'ר שכמה מודולים משתמשים בו הופך למודול

כאשר התשובה לשאלה השנייה או השלישית חיובית, הפיצ'ר הופך למודול עצמאי, וכל המודולים פונים אליו דרך רשימת המותר.

שלוש השאלות הקובעות

1. כמה מודולים צריכים אותו?
2. האם יש לו אילוצים חיצוניים או מצב עצמאי?
3. האם שינוי בו חייב לחול בכמה מקומות בבת אחת?

קידום נעשה במידה: מודול נוסף פירושו תלות נוספת, ולכן שלוש השאלות נבחנות לפני כל קידום.

הכרעה 3: שלושה סוגי מודולים

מודול שירות

משרת מודולים אחרים:
עיצוב, שפה, זהויות,
התראות.

מודול מסך

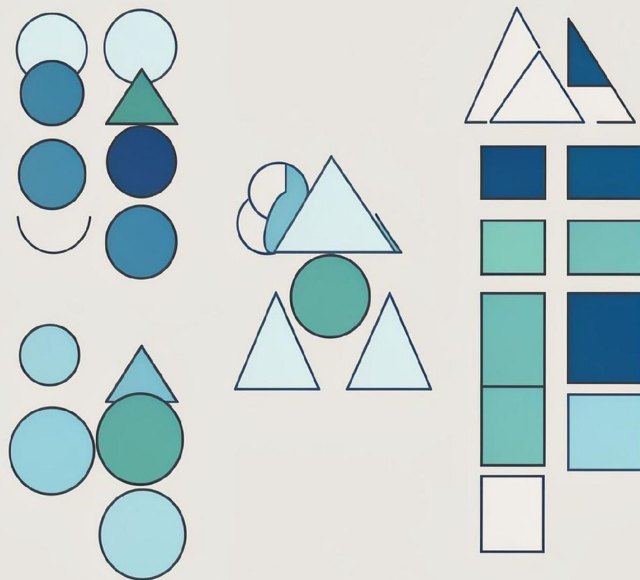
משרת אדם: דיווח, מעקב
ומונה אישי, דשבורד, עוזר
חכם, ניתוח, הגדרות.

מודול תפעולי

משרת את מפעיל המערכת: התקנה, תחזוקה, בדיקה עצמית.

שלושת הסוגים עומדים באותם תנאים:
אחריות אחת, ממשק מוגדר, ויכולת החלפה.
סדר הבנייה נגזר מהתלות:

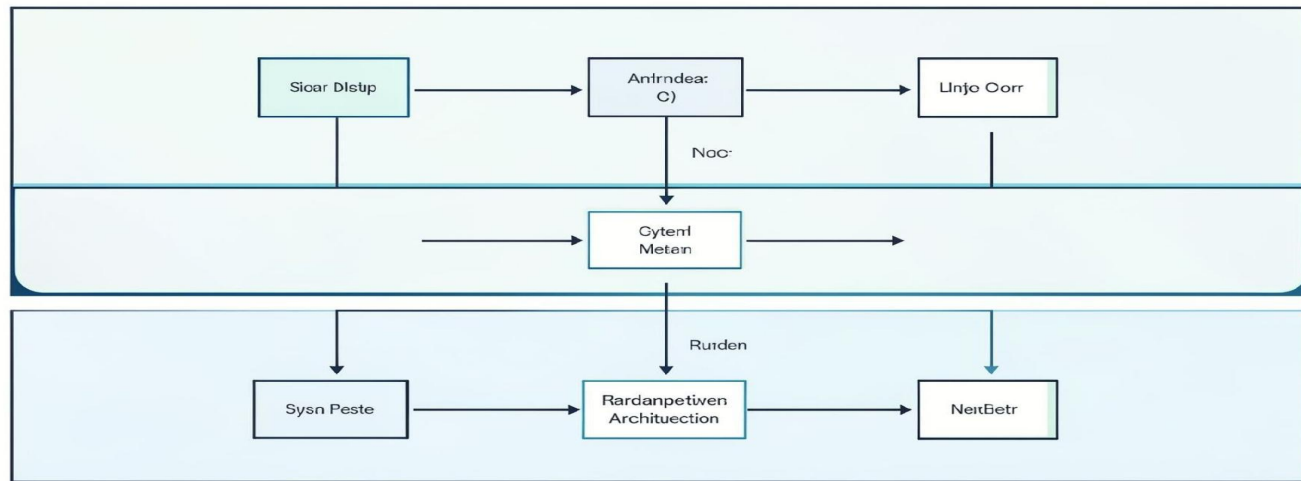
- **מודולי שירות נבנים ראשונים**
- **מודולי מסך אחריהם.**



דוגמה: מודול מסך

1

2



המבחן

- שינוי בכללי ההתחברות נעשה במודול הזהויות בלבד
- כל המסכים יורשים/ מקבלים אותו מיד

התחברות והתנתקות

- מופיעות בכל מסך שייכות למודול הזהויות.
- כל מסך מציג את כפתור ההתחברות
- לוגיקת בדיקת המשתמש יושבת במקום אחד.

במודול כמה פיצ'רים

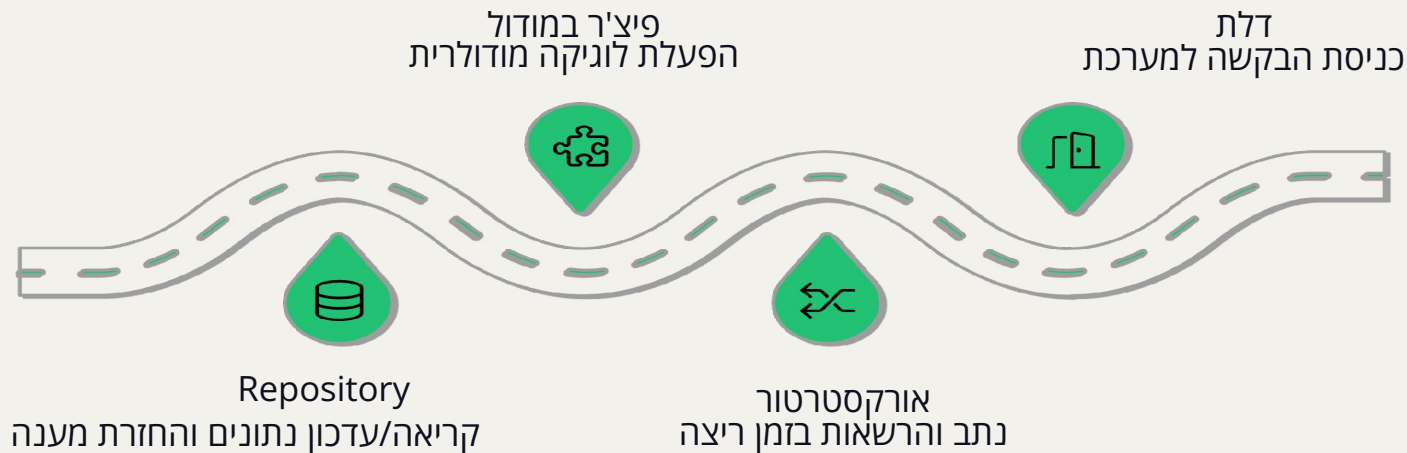
- מודול הדיווח מכיל
- שליחת דיווח
- ניסוח מדיבור חופשי
- צפייה במצב דיווח קיים.

מודול מסך כולל את שלוש השכבות

- מסך בשכבת המציג
- פעולות בשכבת המחליט,
- נתונים בשכבת הזכר.

הכלל: מה שמופיע בכל מקום שייך למודול אחד. ההופעה משותפת, הבעלות יחידה.

מי מנהל מה- שתי רמות: יחידת בניה = המודול, יחידת הניהול = פיצ'ר



מסלול כל בקשה: דלת, אורקסטרטור, הפיצ'ר במודול, Repository, ובחזרה.

ה־Repository מנהל את הגישה לנתונים
כל המודולים פונים דרכו לבסיס הנתונים

בזמן ריצה מנהלים ברמת הפיצ'ר האורקסטרטור מנתב כל בקשה לפעולה מדויקת ברשימת המותר, ובודק הרשאה לכל פעולה בנפרד.

בונים ברמת המודול כל מודול נבנה בשלמותו, עם קבצים משלו, ומוחלף כיחידה.



המחיר של כל חלופה

היישום שבחרנו

1. קובץ נפרד לכל מודול
2. כיבוי פיצ'ר ברשימת המותר
3. שמירת גרסה לפני כל שינוי

כיבוי פיצ'ר תקול באורקסטרטור

האורקסטרטור דוחה כל בקשה אליו בזמן ששאר
המערכת פועלת (שינוי שורה אחת ברשימת המותר).
התנאי

כל הבקשות עוברות בנקודה אחת.

מיקרוסרוויסים

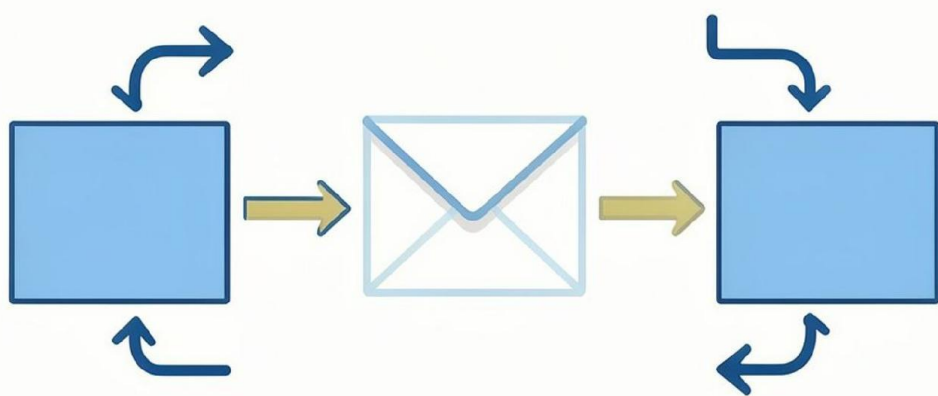
כל פיצ'ר או פנקציה אוטונומית

- תחזוקה שוטפת יקרה
- רדיוס פגיעה צר.
- תיקון נפרס על השירות הבודד.

דוגמה למערכת שבה מיקרוסרוויסים נדרשים

מסחר אלקטרוני בקנה מידה גדול

- בשל הבדלי עומס בין חלקים,
- צוותים נפרדים שמפתחים את המערכת
- מחיר כשל שונה לכל חלק



מעטפת אחידה - שפה אחידה

- | נכנס | יוצא |
|---------------|------------------------|
| • לאיזה מודול | • האם הצליח |
| • איזו פעולה | • הנתונים |
| • מה התוכן | • ואם נכשל - מה השגיאה |
| • באיזו שפה | |

כל עוד המעטפת נשמרת, כל מודול ניתן להחלפה
מודול חדש נכנס בלי לגעת בקיימים

בלי מעטפת אחידה
כל חיבור הוא עבודה מחדש

רשימת המותר: מה המערכת מסכימה לעשות

02

דחייה אוטומטית

פעולה שאינה ברשימה נדחית, גם אם הקוד שמבצע אותה קיים

01

בדיקה לפני כל ביצוע

מקום אחד בודק: האם הפעולה קיימת ברשימת המותר של המודול הזה

03

הכלל הבסיסי

אין פעולה שאין לה מקרה שימוש, ואין מקרה שימוש שאין לו שורה ברשימת המותר

Access control

ALLOWED ITEMS

CHECKLIST



Yet card



Verified ID

BLOCKED ITEMS



Unknown Device



Open don Dor



Untomonized Baze

מהטבלה למערכת, חלק א: המבנה

הכתובת במערכת	בטבלת מקרה השימוש
לאיזה מודול הפעולה נכנסת	שיוך למודול
שורה ברשימת המותר, ופונקציה שמבצעת אותה	היכולת, כלומר הפיצ'ר
קובץ המסך של המודול	שכבה מציג
הפונקציה שמחליטה	שכבה מחליט
דרך נקודת הגישה היחידה לנתונים	שכבה זוכר

מהטבלה למערכת, חלק ב: המימוש

הכתובת במערכת	בטבלת מקרה השימוש
הפונקציה עושה את העבודה בעצמה	דרגת מימוש, קוד רגיל
הפונקציה פונה לשירות ה־AI הנפרד	דרגת מימוש, AI
מה חוזר בשדה השגיאה	חריג ותרחיש כשל



שורה בטבלה שאין לה כתובת בעמודה הימנית פירושה שההחלטה טרם התקבלה.

מקרה שימוש



מהו מקרה שימוש?

מקרה שימוש עונה על שתי שאלות:

מה המערכת עושה, ואיך הפיתוח מתחלק.

חלוקת הפיתוח היא הכרעה שנרשמת במסמך החלטה.

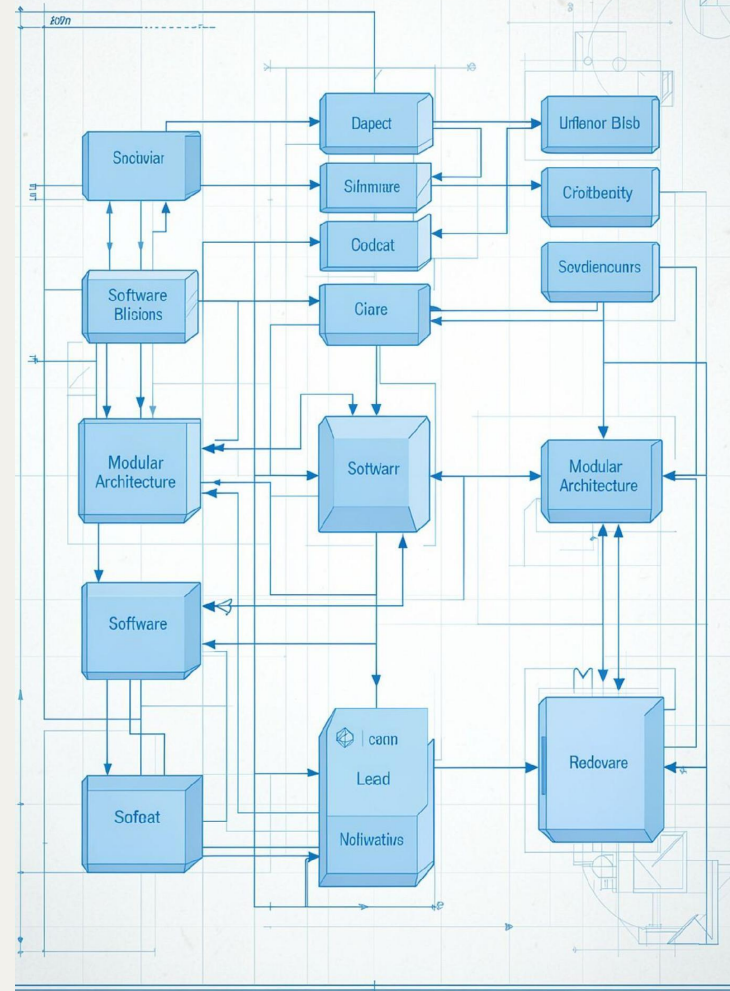
ההכרעה נדרשת לפני תחילת הבנייה

- קובעת את מבנה הקבצים

- את סדר העבודה

- את אופן הבדיקה

Modular Architecture:



מקרה השימוש, ואיך הוא נראה בקוד

ארבעה שדות בכל שורה
מי רשאי לבצע, דרגת
המימוש, סיפור
המשתמש שממנו היא
נגזרה, והאם היא כבר
עובדת

מבנה הטבלה
טבלת מקרי השימוש
מנוהלת בקובץ אחד
במערכת, שורה לכל
יכולת

שמונה עשרה שורות
כל אחת מהן הגיעה מהטבלה שכתבנו

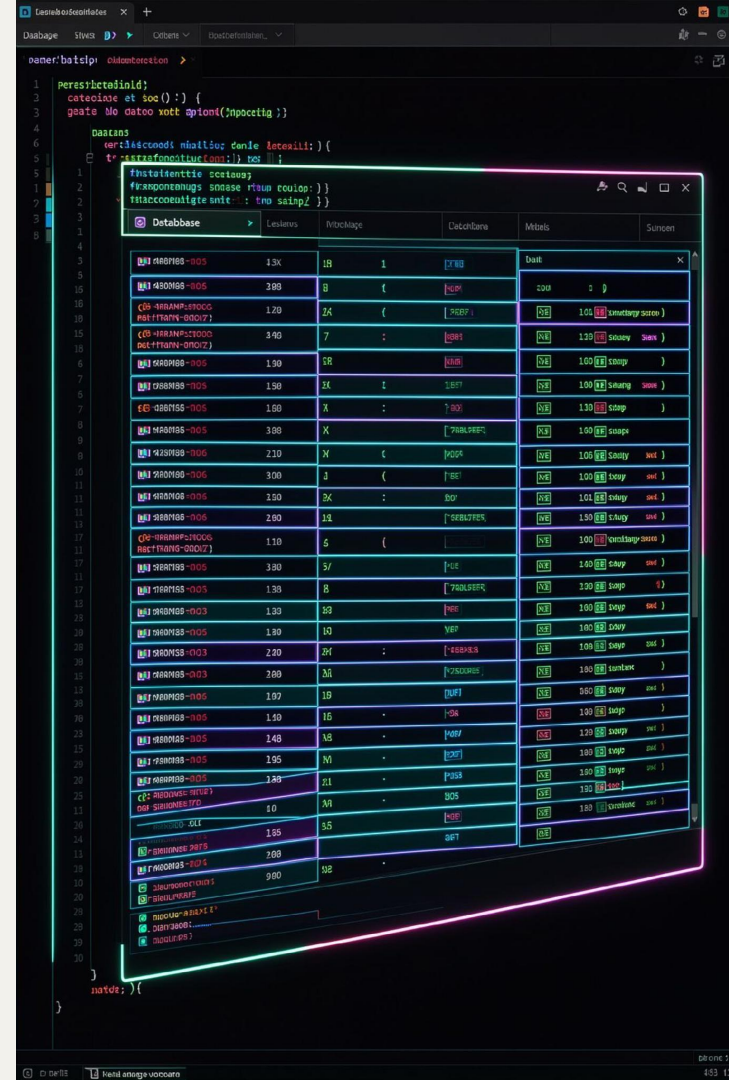


הידע שייך לכם

מה קורה כשמבקשים
פעולה שאינה ברשימה
יוצרים לה סיפור
משתמש!

הבקרה מתרחשת
ומטופלת בשכבה
הראשונה- בלוגיקה
העסקית.

יצרתם את הטבלה, ולכן
אתם יודעים בדיוק מה
המערכת מסוגלת
לעשות.

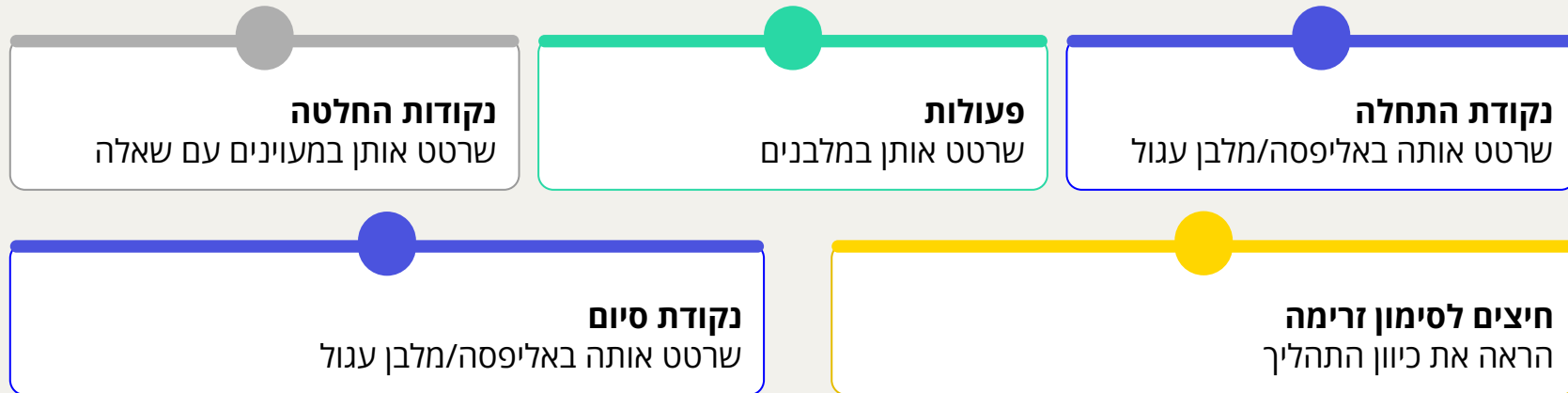




תרשים זרימה (Flowchart)

מטרת הדיאגרמה: להציג את הזרימה הלוגית של תהליך במערכת, צעד אחר צעד

רכיבי חובה בדיאגרמה:



מהתרשים לבניית המערכת: קוד רגיל

בשלב האפיון שרטטנו תרשימי זרימה.
הם משתמשים למפרט של המערכת

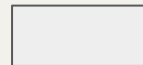
קוד רגיל הוא תרשים, מנוסח כהוראות מפורשות.
אין בו שיקול דעת, כל קלט מחזיר תמיד את אותה
תוצאה



כל חץ
הוא הסדר



כל מעוין
הוא תנאי



כל מלבן
בתרשים הוא פעולה

מתי התרשים מספיק, ומתי נדרש AI

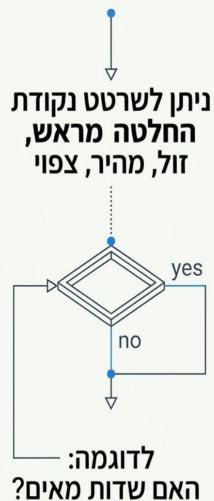
המבחן: האם ניתן לשרטט את נקודת ההחלטה מראש

- ניתן לשרטט, זהו קוד רגיל - זול, מהיר וצפוי
- לא ניתן, התשובה תלויה בטקסט חופשי? זה AI

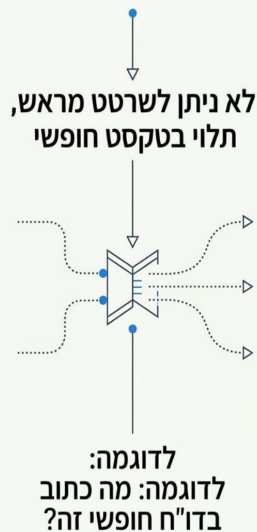
"האם השדות מלאים?" מעוין הוא אפשרויות ההחלטה.
"מה נאמר בדיווח החופשי הזה" איננו מעוין

כל נתיב חייב להוביל לנקודת סיום.
חייבים תמיד לקבוע נתיב סיום ונקודות יציאה

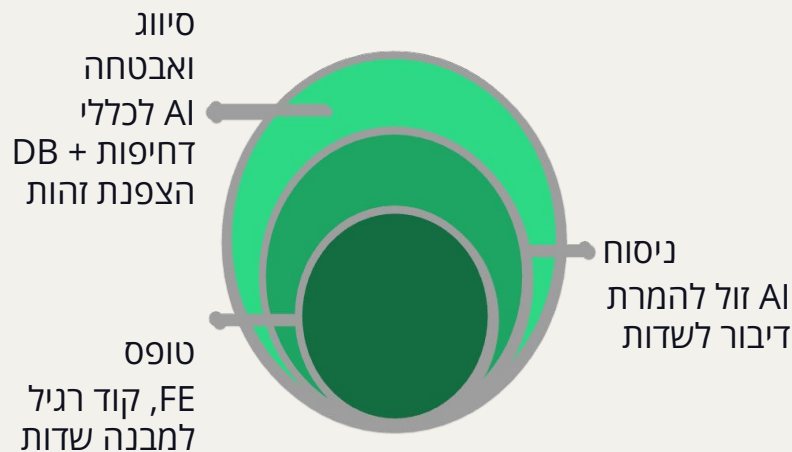
קוד רגיל



בינה מלאכותית



דוגמה: דיווח תקלה מפורק להחלטות



ההחלטות שורה אחר שורה

טופס הדיווח: קוד רגיל, FE. מודול אחד

ניסוח הדיווח מדיבור חופשי לשדות מובנים: AI, זוהי משימת ניסוח, לא הסקה לכן מספיק מודל זול.

סיווג דחיפות: AI עם כללי גבול בקוד רגיל. תקלה קריטית לעולם לא תלויה במודל.

שמירת הדיווח והפרדת הזהות המוצפנת: קוד רגיל, DB, דרך Repository אחד.

כל שורה כזאת היא החלטת פיתוח מתועדת.

דרישה 1 מקצה לקצה: אנונימיות המדווח

שש יכולות, וכולן קוד רגיל

הזהות מופרדת
הדיווח נשמר עם
טוקן
הקישור לזהות
בשכבה נפרדת

ברשימת המוטר
שליחת דיווח,
שליפת מצב
דיווח

המודול
דיווח

החריג
מנהל מנסה לחשוף זהות: הפעולה לא קיימת ברשימת המוטר
שלו, ולכן נדחית ונרשמת



האנונימיות היא סדר פעולות, לא הצהרת כוונות עסקית

אפשר לכתוב "המערכת אנונימית" במסמך. זה לא עושה דבר בקוד.

הקישור לזהות

יושב בטבלה נפרדת ומוצפנת -
מנותק מבחינה ארכיטקטונית
מנתוני הדיווח עצמו.

בטבלת הדיווחים אין עמודת
שם

לא כי היא ריקה, אלא כי היא
לא נוצרה. ההיעדר הוא חלק
מתכנון המערכת, לא מקרי.

מה שעושה את ההבדל

הטוקן נוצר לפני שהדיווח
נשמר, והדיווח נשמר עם
הטוקן בלבד. זהו סדר
הפעולות שמבטיח אנונימיות
אמיתית.

החריג שכתבנו בטבלה

הוא זה שקובע שהחשיפה לא
תיכנס לרשימת המותר של מנהל.
הגבול הוגדר מראש, בכתב.

החשיפה היא פעולה נפרדת

דורשת הרשאה ייעודית, ונרשמת
ביומן גם כשהיא נדחית.
אין חשיפה שקטה.

מה נדרש לייצור



חתימה

זיהוי שינויים בלתי
מורשים בנתוני
הזהות המוצפנים



החלפה תקופתית
רוטציה מתוזמנת של
המפתח כדי לצמצם
חלון חשיפה



מפתח מחוץ לפרויקט
אחסון המפתח
בשירות ייעודי, מנותק
מבסיס הקוד



אלגוריתם תקני
יישום שיטת ההצפנה
בתקן מוכר ומבוקר

מה שעובד, מול מה שבייצור

מה עובד כעת



שכבת הזהויות מוצפנת, ומי שפותח את הגיליון רואה תווים ולא שמות.

והאם זה מספיק לייצור? לא תמיד יש לבדוק



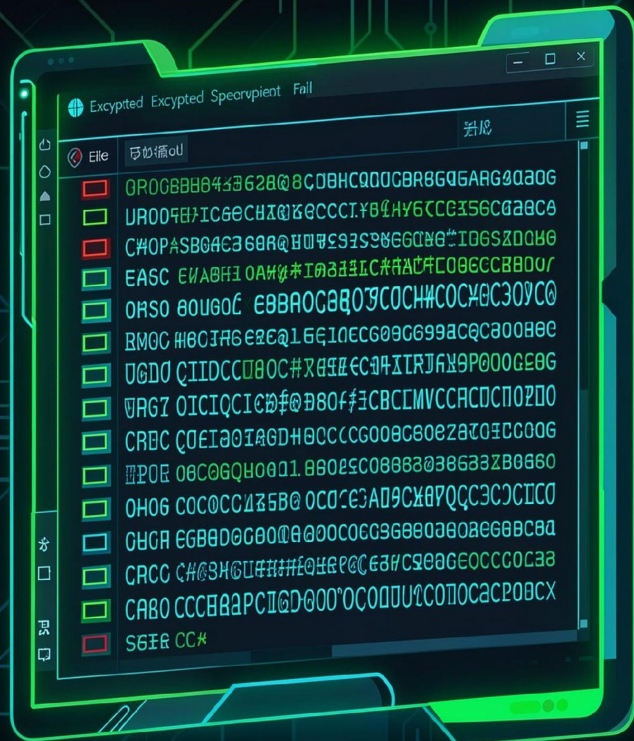
המפתח יושב
באותו פרויקט
מי שרואה את הקוד
רואה גם אותו



אין זיהוי שינוי
מי שעורך את
הגיליון יכול להחליף
ערך, והמערכת לא
תדע



השיטה חלשה
כשכל הכתובות
בארגון נגמרות
באותה סיומת, יש
נקודת אחיזה
לפיצוץ



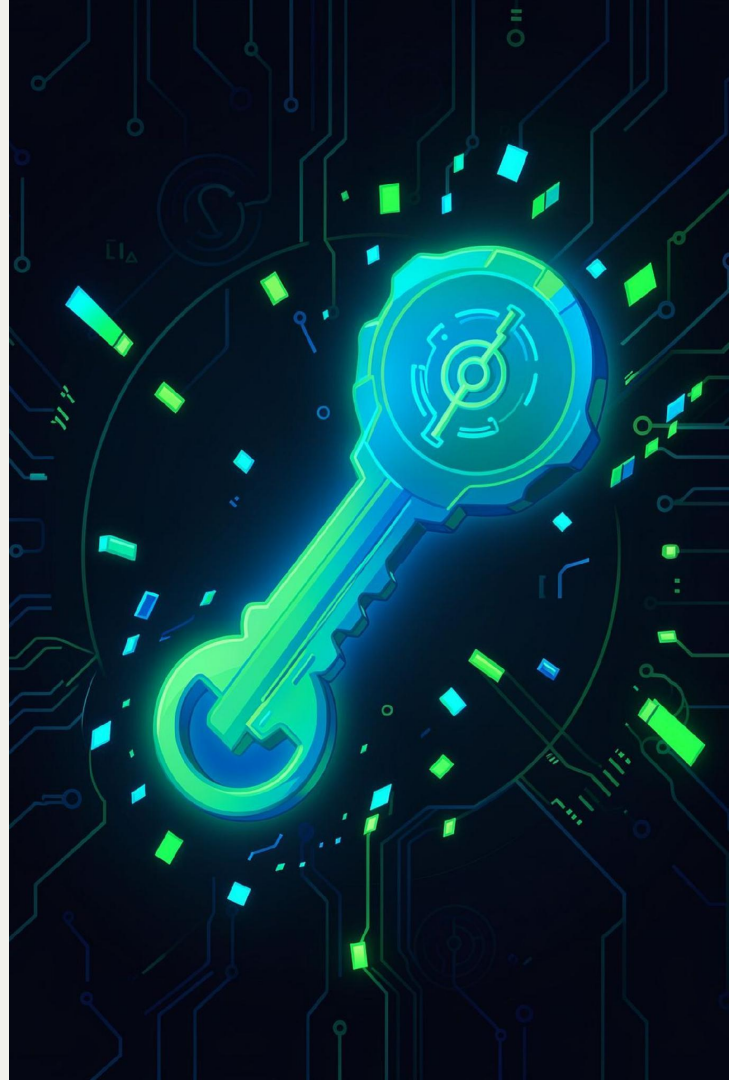
ההצפנה

בטבלת הזהויות שמור הקישור בין הטוקן לבין מי שדיווח.
הוא חייב להיות הפיך, כי בעל הרשאת האנונימיות צריך
לפעמים לחשוף אותו. לכן זו הצפנה.

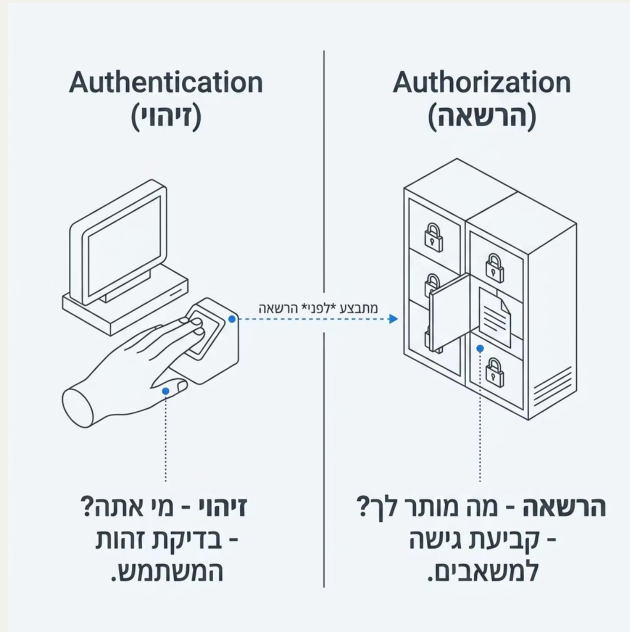
ערבול על מפתח: המידע מושתת על מפתח סודי, ומי שפותח
את הגיליון רואה רצף תווים ולא כתובת מייל.

מה זה כן משיג: מנהל שנכנס לגיליון, בכוונה או במקרה, לא
רואה שמות.

זה מכסה את רוב מקרי החשיפה האמיתיים בארגון, שרובם
המכריע הם חשיפה מקרית ולא תקיפה.



ניהול משתמשים: ההבחנה המקצועית

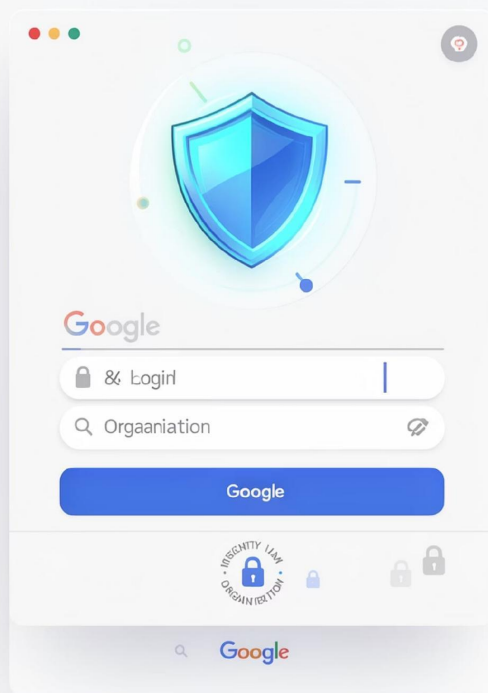


כלל הברזל

את הזיהוי לא בונים לבד.

סיסמאות בגיליון = הפרצה הכי מסוכנת שיש.

את ההרשאה כן בונים, כי היא הלוגיקה העסקית שלנו
מי רשאי לעשות מה במערכת.



הזיהוי מגיע בחינם



הקוד שואל מי
מחובר
ומקבל מייל
מאומת בחזרה



גוגל דורשת
התחברות
מכל משתמש,
אוטומטית



בפרסום ה-Web
App
בוחרים "רק"
משתמשי הארגון"



ללא סיסמאות

ללא מסכי הרשמה.
הזהות הארגונית מנוהלת בארגון באבטחת המידע.

ההרשאה בגיליון

לשונית users

שלוש עמודות פשוטות שמגדירות את כל מערכת ההרשאות:

email	role	status
user@org.com	admin	active
staff@org.com	viewer	active
old@org.com	viewer	inactive

הזרימה

גוגל מזהה ← הקוד שולף תפקיד מהטבלה ← מחליט מה מותר.

מי שלא קיים בטבלה לא נכנס.

זהן המימוש של "למי מותר לעשות מה" של מקרה השימוש.

הגיליון הוא מסד הנתונים של ההרשאות - פשוט, שקוף, וניתן לעריכה.

החיבור למערכת הדיווח

הערוץ המזוהה

הזיהוי משרת את הערוצים המזוהים - המשתמש מחובר, הקוד יודע מי הוא.

הערוץ האנונימי

הערוץ האנונימי שובר את הקשר במכוון: בדיווח נשמר רק טוקן המדווח, והקישור לזהות מוצפן בשכבה נפרדת.

ניהול המשתמשים הוא המקום שבו הבטחת האנונימיות פוגשת את הקוד.



ערוץ מזוהה: משתמש באימות של גוגל ומקשר ישירות את הזהות.



ערוץ אנונימי: שומר רק טוקן דיווח, הזהות מוצפנת בשכבה נפרדת.

שלב 0: תשתית

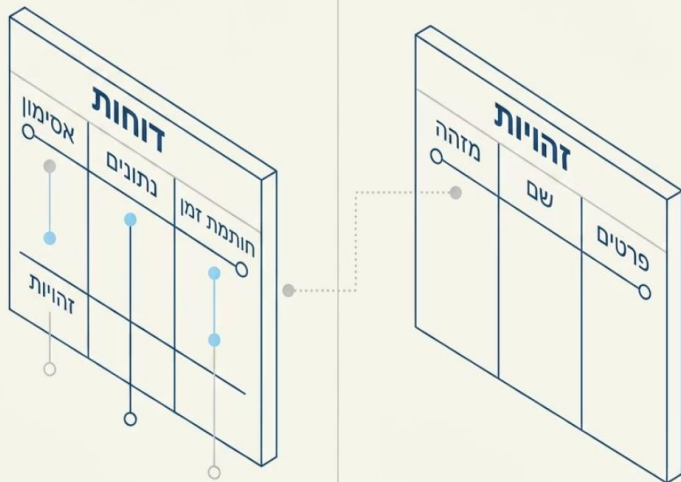
ניצור פרויקט Apps Script חדש
בחשבון ה Google שלנו

תוך דקות בנינו סביבת עבודה מלאה:
עורך, אירוח, גרסאות, כספת.



שלב 1: ה-DB

נתחבר לגיליון עם שתי לשוניות לפי ה-ERD
ניצור אותו עם CLAUDE



reports

עם טוקן מדווח, לא
שם - הפרדת הזהויות
מהשורה הראשונה.

identities

בנפרד - הפרדה
נקייה בין זהות
לדיווח.

שלב 2: ה-BE

פרויקט Apps Script עם שלושה רכיבים:

doPost

מקבל דיווח וכותב
לגיליון.

קריאה ל- KEY

מהשרת - לא מהדפדפן.

מפתח נשמר ב-Script Properties
מאובטח בצד השרת.

ה-KEY הסודי עוזב את הדפדפן ולעולם אינו מופיע שם.





שלב 3: החיבור

וה-FE מוגש ישירות מ-Apps Script (HtmlService):

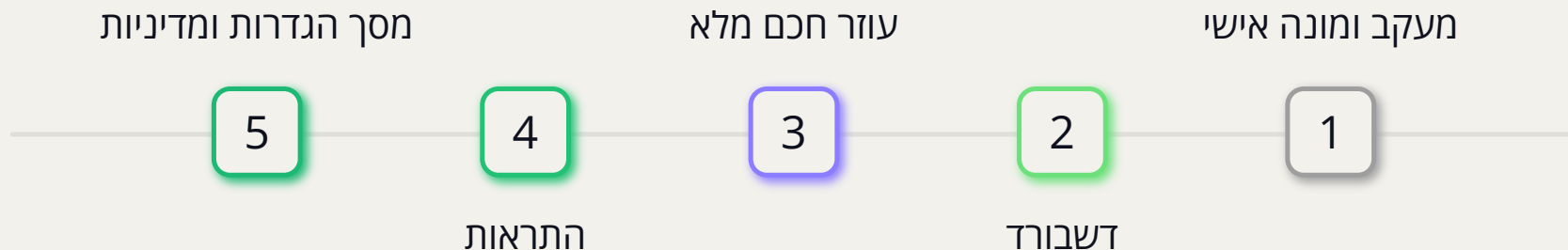
אפס כלים חיצוניים

מסך ומוח יחד

אותה כתובת

שלב 4: הרחבה מודול אחר מודול

כל מודול ייבנה באותו דפוס, על שלד שכבר עובד:



לכל צעד Commit

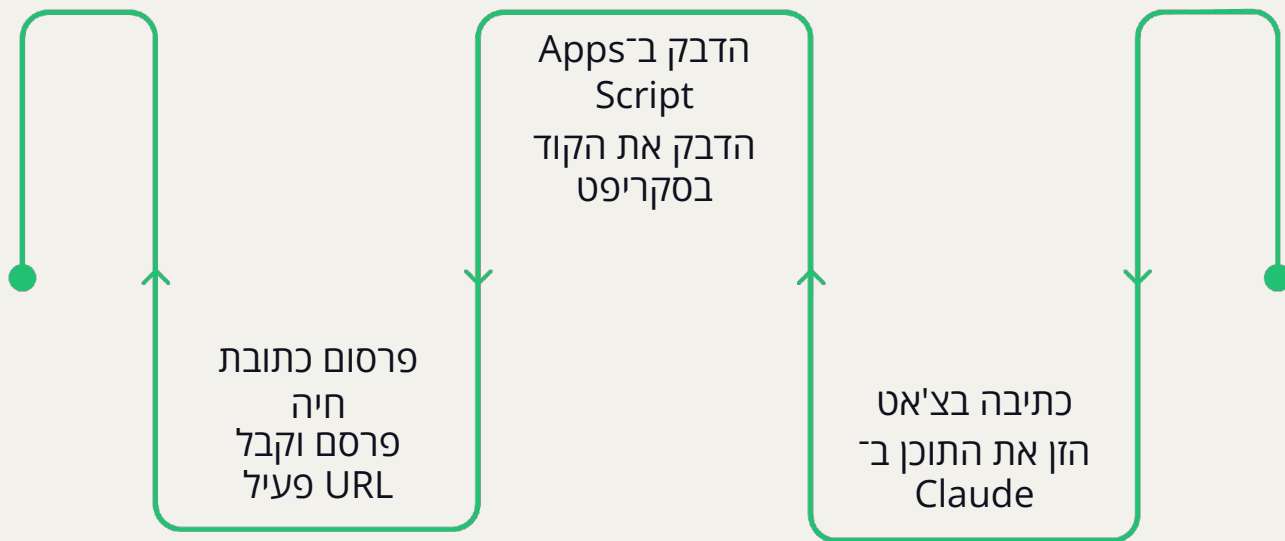
רשת ביטחון לכל שינוי.



גישה דרך ה-Repository בלבד

הפרדה נקייה בין שכבות.

המסלול: מהצ'ט לכתובת חיה



שני כלים, אפס טרמינל. האדם הוא הידיים, הצ'ט הוא המוח, והתלמיד מבין כל צעד כי עשה אותו בעצמו.

צעד 2: מבקשים מקלוד בצ'ט

משפט ההקשר הקבוע

"הקוד ירוץ ב-

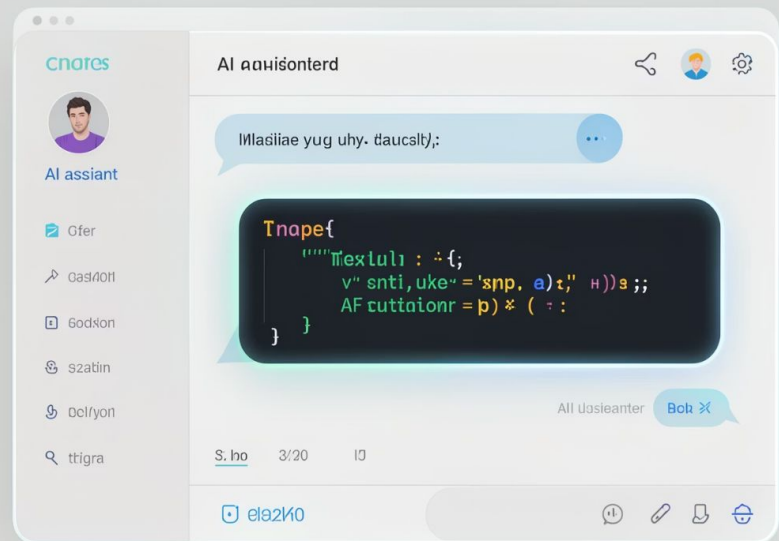
Web App Google Apps Script ויוגש כ-

הפרד קובץ HTML (תצוגה) מקובץ Code.gs

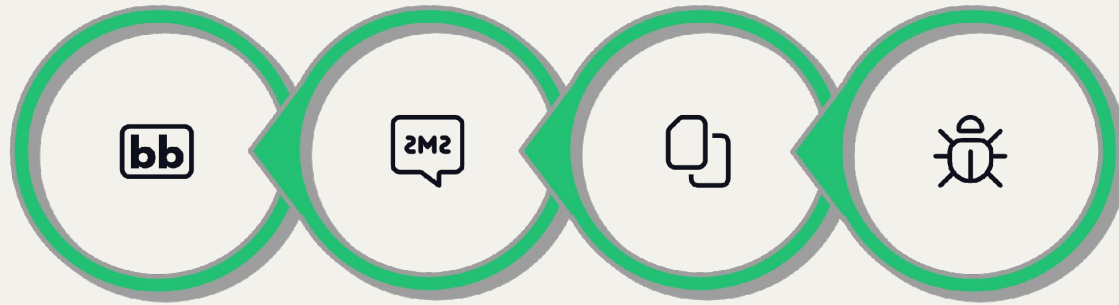
(לוגיקה)"

צור הפרדה בין השכבות, המודולים והפונקציות

ההפרדה מוגדרת מראש בפרומפט - כך קלוד יודע בדיוק איך לבנות את הקוד, ומקבלים פלט מסודר ומוכן להדבקה.



וכשמהו לא עובד



הדבק ופרוס
מחדש

קבל תיקון

העתק הודעת
שגיאה

שגיאה מופיעה

הלולאה: בקשה ← הדבקה ← שגיאה ← תיקון ← הדבקה
אתם מנהלים את הסוכן, לא מקלידים או יוצרים קוד.

איך יודעים שקיבלנו את מה שהזמנו

2
שאלה שנייה האם קיים במערכת משהו שאינו בטבלה שלי

1
שאלה ראשונה האם כל שורה בטבלה שלי קיימת במערכת ועובדת

מריצים אחרי כל סבב בנייה, לא בסוף

⊗ השנייה היא המסוכנת: פעולה שאיש לא ביקש היא הכרעה שהתקבלה במקומנו

עכשיו תורכם: בונים פרוסה

01

בחרו דרישה אחת

זו שבלעדיה המערכת חסרת ערך

02

מלאו את הטבלה

יכולת, שכבה, דרגת מימוש, נימוק, וחריג אחד

03

תרגמו לרשימת המותר

מודול, פעולה

04

בנו את הפרוסה

מהמסך ועד הגיליון, ופרסמו

05

הריצו את שתי שאלות הבדיקה

מה חסר, ומה נוסף בלי שביקשנו

מה יוצא מהמפגש

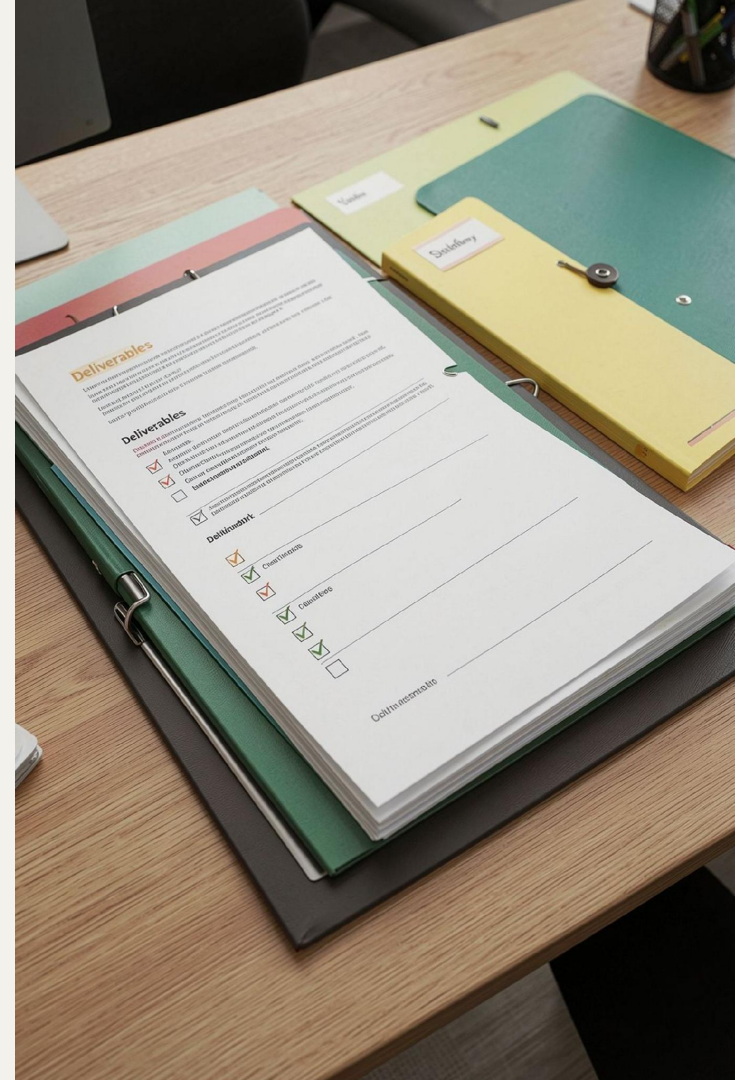
כתובת שעובדת
של פרוסה אחת

טבלת מקרה השימוש
שממנה היא נבנתה

רשימת המותר
שנגזרה ממנה

ממצאי הבדיקה
מה חסר, ומה נוסף בלי שביקשנו

הכול נשמר לתיקיית הפרויקט לפי נוהל השמות



רטרוספקטיבה

שלוש שאלות לסיום

3

מה אקח איתי כבר מחר?

פעולה קונקרטית אחת שאוכל
ליישם מחר בבוקר בעבודה.

2

מה הפתיע אותי?

משהו שלא ציפיתי לו - תפיסה
שהתהפכה, הנחה שנשברה.

1

מה למדתי היום שלא ידעתי?

תובנה חדשה שמשנה את
האופן שבו אני חושב על ניהול
מוצר.

תודה רבה

רילי לבבי

053-4365200

levavir@gmail.com

